

Ejercicio: 2 de unidad 2

1. **SQLi** (Porque usa el truco de ' OR ' 1 ' = ' 1 para entrar a la base de datos).
 2. **XSS** (Porque hace aparecer alertas en la pantalla con código).
 3. **CSRF** (Porque engañan al usuario para que haga un movimiento de plata sin querer).
 4. **Autenticación Rota** (Porque guardar claves en texto plano es un error de seguridad básico).
 5. **XSS** (Porque deja meter código JavaScript en un comentario).
-

Ejercicio: Matching

- **SQL Injection** b) Prepared Statements / Parametrized Queries
 - **XSS** c) Escapar salida + Content Security Policy
 - **CSRF** a) Usar tokens Anti-CSRF
 - **Broken Authentication** d) Hashing fuerte + MFA
-

Ejercicio: Código Vulnerable

1. ¿Qué vulnerabilidad tiene? Tiene **SQL Injection**.
2. ¿Cómo se puede explotar? Se puede explotar poniendo código de SQL en el campo del usuario, por ejemplo, usando ' OR ' 1 ' = ' 1 para entrar sin saber la contraseña o para borrar la tabla de usuarios.
3. ¿Cómo lo corregirías? (versión segura) En lugar de sumar textos con el +, hay que usar parámetros para que el sistema no confunda el nombre con un comando:

`cursor.execute("SELECT * FROM usuarios WHERE usuario = ?", (usuario,))`